

Assignment 1: MCP Fundamentals

Theory and FastMCP Documentation Answers

October 1, 2026

1 Theory Questions

Q1. Tool vs. Resource vs. Prompt

The three primitives differ mainly in *who decides when they are used*, and whether they cause side effects.

	Tool	Resource	Prompt
Controlled by	The model decides to call it mid-conversation	The client/application decides to pull it into context	The user picks it deliberately (e.g. a slash command)
Nature	An action: takes arguments, runs code, may have side effects	Read-only data identified by a URI; no side effects	A reusable instruction template with arguments, returning messages
Returns	A result for the model to reason over	Content (text/JSON/binary) at that URI	One or more messages that start or steer a conversation

One-line examples.

- **Tool:** reach for it when the model must *do* something, e.g. `add_expense(amount=450, category="food")` while the user says “I just spent 450 on lunch”.
- **Resource:** reach for it when you want to expose *data*, e.g. `expenses://summary/2026-09` to hand the model a monthly summary as context.
- **Prompt:** reach for it when a person wants a *repeatable workflow*, e.g. a “Monthly budget review” template they select on purpose.

Q2. Why does a local MCP server talk to Claude Desktop over stdio?

What it means in practice. Claude Desktop does not connect to the server over a network. It *launches the server as a child process* (using the command from its config, e.g. `python server.py` or `uv run server.py`) and exchanges MCP’s JSON-RPC messages through the process’s standard streams: the client writes requests to the server’s `stdin`, and the server writes responses to its `stdout`. There is no port, URL, or HTTP stack involved. Two consequences follow:

- **stdout is the protocol channel.** A stray `print()` corrupts the JSON stream, so logs must go to `stderr`.

- **Lifecycle is owned by the client.** The server starts when the client needs it and exits when the client closes the pipe. It does not “stay running” on its own.

Why it makes sense for local use.

- **No network exposure.** Nothing listens on a port, so no other machine (or local process) can connect to it. This is a smaller attack surface than a local HTTP server.
- **No auth to build.** The process runs under the user’s own account and inherits their permissions, and only the parent client holds the pipes, so no tokens, TLS, or CORS handling are needed.
- **Zero configuration.** No port conflicts or firewall prompts; one config entry is enough.
- **Natural fit for local resources.** Local files, SQLite databases, and CLI tools are exactly what a desktop assistant needs, and a per-session subprocess keeps state simple and isolated.

The trade-off is that stdio serves a single client per process and cannot be reached remotely, which is where HTTP comes in (next section).

2 FastMCP Documentation Questions

Q1. stdio vs. http, and what changes in mcp.run()

FastMCP’s deployment docs describe three transports.

	stdio (default)	http (Streamable HTTP)
Process model	Client spawns a new server process per session and manages its lifecycle	One long-running web service reachable at a URL (default <code>http://localhost:8000/mcp</code>)
Clients	One client per process	Many clients at once
Reach	Same machine only	Local or remote deployment
Typical use	Claude Desktop, Claude Code, local dev tools, personal servers	Shared/team servers, cloud deployment, anything needing auth or scaling

When I would pick which. I would pick **stdio** for a personal, local server used by one desktop client: it is the simplest option and needs no networking or auth. I would pick **http** when the server must be shared by multiple users or clients, run on another machine, or sit behind authentication and standard web infrastructure. The **sse** transport is the original HTTP-based transport; the docs treat it as legacy and recommend Streamable HTTP for new work, so SSE is only worth using for compatibility with older clients.

What changes in code. Only the argument to `run()`. The tools, resources, and prompts stay identical.

```
from fastmcp import FastMCP

mcp = FastMCP("my-server")

# ... @mcp.tool / @mcp.resource / @mcp.prompt definitions ...

if __name__ == "__main__":
    mcp.run()                                # stdio (
    default)
```

```
# mcp.run(transport="http", host="127.0.0.1", port=8000) #
  Streamable HTTP
# mcp.run(transport="sse", host="127.0.0.1", port=8000) # legacy
  SSE
```

The transport can also be chosen from the CLI without editing code, e.g. `fastmcp run server.py -transport http -port 8000`.

Q2. Plain string vs. dict/Pydantic return values (structured output)

Plain string. A tool that returns `str` produces *unstructured* output: a text content block. The client only receives text, and any meaning inside it (numbers, IDs, status) is left for the caller to parse.

Dict or Pydantic model. Object-like return values (dict, Pydantic BaseModel, dataclass) produce *structured output*. FastMCP derives a JSON output schema from the return type annotation and returns the data as machine-readable *structured content*, alongside a text/JSON serialization for clients that only read text. The structured result is validated against the schema, and Pydantic models additionally give typed fields, descriptions, and nested types. Primitive annotations (e.g. `int`, `str`) are not objects, so FastMCP wraps them under a `result` key when it emits structured content.

```
from pydantic import BaseModel

@mcp.tool
def add_text(a: int, b: int) -> str:
    return f"Total is {a + b}"          # client sees only: "Total is
    7"

class Total(BaseModel):
    total: int
    currency: str

@mcp.tool
def add_structured(a: int, b: int) -> Total:
    return Total(total=a + b, currency="INR")  # client gets {"total":
    7, "currency": "INR"}
```

Why it matters to a client.

- **Reliable parsing.** A programmatic client can read `result.data["total"]` directly instead of using regexes on prose, which breaks as soon as the wording changes.
- **A contract.** The output schema is advertised with the tool, so clients know the response shape before calling and can validate it or generate UI/types from it.
- **Safe evolution.** Adding a field to a model does not break consumers, while rewording a sentence in a string can.
- **Trade-off.** For output the LLM simply reads (a short explanation or a message), a string is fine and often more natural; structured output pays off when code, UIs, or other tools consume the result.

Sources. FastMCP docs: *Running Your Server* (<https://gofastmcp.com/deployment/running-server>) and *Tools* (<https://gofastmcp.com/servers/tools>); MCP specification, transports section.